IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 16 March 2026, accepted 14 May 2026, date of publication 25 May 2026, date of current version 4 June 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3696556

RESEARCH ARTICLE

OMARF: An Orchestrated Multi-Agent Resilience Framework for Autonomic Software-Defined Offshore Wind Industrial Networks

AGRIPPINA MWANGI[1], MIKKEL GRYNING[2], ELENA FUMAGALLI[1], AND MADELEINE GIBESCU[1], (Member, IEEE)

[1]Copernicus Institute of Sustainable Development, Utrecht University, 3584 CB Utrecht, The Netherlands
[2]System Design Specialists, Ørsted, 2820 Gentofte, Denmark

Corresponding author: Agrippina Mwangi (a.w.mwangi@uu.nl)

This work was supported by European Union's Horizon 2020 Research and Innovation Programme under the Innovative Tools for Cyber-Physical Energy Systems (InnoCyPES) Project through the Marie Skłodowska-Curie Actions Under Grant 956433 and Ørsted Research and Development.

ABSTRACT Software-defined industrial networks face growing challenges from component failures, network congestion, thermal instability, and cyberattacks, requiring resilience mechanisms that can respond and adapt in real time. This paper presents the Orchestrated Multi-Agent Resilience Framework (OMARF), an autonomic framework that coordinates self-healing and self-defense agents across the control, data, and knowledge planes of software-defined industrial networks. Unlike conventional approaches that treat resilience and security separately, OMARF integrates cloud-native controller recovery, reinforcement-learning-based traffic adaptation, and adaptive cyber defense within a unified orchestration framework. The framework continuously monitors multidimensional network states, detects policy violations, and activates the most appropriate mitigation agents in real time. Simulations conducted in an offshore wind power plant network demonstrate that OMARF reduces path latency by up to 28.3% while improving throughput across diverse fault and attack scenarios. In degraded operating conditions, the framework maintained stable adaptive behavior, highlighting its potential to improve the reliability, scalability, and cyber resilience of next-generation industrial software-defined networks.

INDEX TERMS Autonomous networking, SDN, offshore wind, network resilience, multi-agent systems, reinforcement learning, autonomic computing.

I. INTRODUCTION

Building resilience is crucial in emerging autonomic software-defined industrial networks that support offshore wind power plants (WPPs), in which system operational reliability must be maintained. These industrial networks support essential machine-to-machine interactions that form the backbone of wind farm monitoring, control, and coordination [1]. As these networks continue to grow in scale and complexity, they are increasingly exposed to unpredictable and often extreme stochastic stressors. These events often give rise to brief network disruptions or, in some cases, prolonged outages that propagate through the system, leading to serious cascading consequences, such as wind turbine misalignment, inaccurate fault isolation, overload conditions, and, in severe instances, unplanned WPP shutdowns or loss of synchronization with the transmission grid [2].

In offshore WPP industrial networks, predictable stressors include scheduled maintenance, seasonal weather patterns, bandwidth congestion during peak data transmission periods, device restarts owing to firmware updates, and known vulnerabilities awaiting patches. In contrast, unpredictable disruptions include evolving cyber threats [3]; sudden failures of Ethernet switches, links, or controllers, configuration corruption from undetected software bugs, and increased network latency due to unexpected surges in WPP-related activity.

WPP industrial networks were built using deterministic communication systems designed for predictable and periodic control workloads. However, these fixed-schedule

The associate editor coordinating the review of this manuscript and approving it for publication was Feng Wu.

networks are unlikely to meet the future demands for reliability, safety, and low latency. As WPPs become more complex, the volume and variability of network traffic will increase sharply. Expanding deployments of IoT sensors, condition-monitoring devices, and edge-computing nodes will generate data loads far beyond the capacity of current deterministic systems. In addition, these next-generation WPPs will include high density turbines, floating platforms, and advanced predictive maintenance systems. This will produce bursts of high-frequency telemetry, real-time environmental data, and control messages. In addition, operational events, such as turbine faults, yaw or pitch anomalies, converter trips, or rapid wind fluctuations, often trigger sudden surges in network traffic. Consequently, network failures can disrupt turbine control, reduce power output, and compromise overall plant safety.

Achieving network resilience in this context extends beyond conventional risk management approaches, which often fail to anticipate the complexity and uncertainty of large, integrated systems. Ganin et al. [4] identified four key capacities that resilience frameworks adopted in these networks must support: (i) planning for adverse events, (ii) absorbing stress, (iii) recovering, and (iv) predicting and preparing for future stressors and disruptions to adapt in the face of uncertainty. Current resilience strategies are often siloed and lack autonomy, limiting their effectiveness under multi-event or extreme conditions, hampering adaptive responses, and rendering systems vulnerable to cascading failures across the network [5], [6].

Recent studies [7], [8], [9], [10], [11], [12], [13], [14], [15], [16] have developed autonomic architectures to address these limitations. Early approaches focused on policy-based automation, which enabled networks to proactively enforce Quality of Service (QoS) and flow rules. Later studies introduced cognitive and self-managing systems that optimize routing and resource allocation using learning algorithms. More recent designs and implementations use agent-based and cooperative frameworks, in which network functions act as independent decision-making units that adapt collectively. These frameworks have evolved to integrate AI-driven orchestration based on learning algorithms, including large language models, to translate high-level intents into automated network policies. This progression highlights how network and service management have evolved to support adaptive, resilient operations in complex and dynamic environments.

Benchmarking these studies and novel reference architectures, this study presents an orchestrated multi-agent resilience framework (OMARF) for autonomic software-defined industrial networks. This framework comprises: (i) *"A Component Recovery Self-Healing Agent"* that is designed to address control plane and data plane component failures, (ii) *"A threshold-Triggered Deep Q-Network Self-Healing Agent"* designed to mitigate Ethernet switch thermal instability under dynamic offshore temperature conditions and intermittent network congestion, and (iii) *"An Event-driven OpenFlow Random Host Mutation (OF−RHM) Self-Defense Agent"* that detects and mitigates both early-stage and late-stage cyberattacks, limiting lateral threat propagation and safeguarding the WPP's critical infrastructure assets.

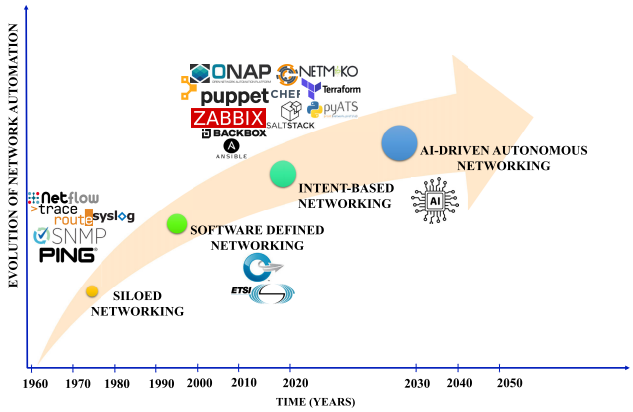


FIGURE 1. The evolution of network automation for network and service management [6].

To deliver continuous and real-time insights into how effectively the proposed OMARF sustains network performance in compliance with IEC 60870-5, IEC 61850-5, IEC 62439-4/5 standards [17], and ASHRAE QoS/SLA guidelines [18], the study modeled the resilience factor of OMARF as a discrete-time dynamic system model to capture its evolving behavior under uncertainty and agent interactions. Based on this model, the study empirically assesses two complementary resilience metrics: the *Health Score* $\mathcal{H}(t)$, reflecting operational integrity, and the *Risk Score* $\mathcal{R}(t)$, indicating vulnerability to disruptions and observing network performance and behavior using a network emulator testbed.

The remainder of this paper is organized as follows: Section II reviews recent resilience frameworks, outlines key performance indicators, and positions the proposed OMARF relative to the current research. Section III presents the proposed OMARF, detailing its operational principles and the integration of orchestrated self-healing and self-defense agents. Section IV describes the experimental setup and methodology, including the simulation environment, parameter configurations, fault-injection methods, and evaluation procedure. Section V presents the experimental results, analysis, and discussion. Finally, Section VI concludes the study and identifies potential directions for future research.

II. STATE-OF-THE-ART AND RELATED WORKS

Industrial network and service management has gradually moved away from siloed, distributed approaches toward more intent-driven, AI-enabled autonomic systems [19], as illustrated in the evolution chart in Figure 1. In modern architectures, industrial network operators define high-level policies and intents that guide the network to meet specific performance indicators and QoS targets [20], [21]. As this evolution continues, AI-enabled autonomic solutions are

TABLE 1. Comparative summary on autonomic and agent-based sdn management frameworks and architectures.

Source	Year	Study	Key Contribution	Approach	Application Domain	Testbed
Bari *et al.* [7]	2013	PolicyCop	Autonomic QoS policy enforcement framework for SDN	Uses SDN northbound API for SLA enforcement; includes Policy Validator and Policy Enforcer modules	SDN QoS management (OpenFlow)	5 OVS + 4 hosts connected by GRE tunnels; Floodlight controller
Tsagkaris *et al.* [8]	2015	AutoFlow	Policy-driven autonomic management for multi-domain SDN	Integrates ANM policy layer into SDN control plane for adaptive policy enforcement	Carrier-grade multi-domain SDN	GÉANT OpenFlow testbed
Neves *et al.* [9]	2016	SELFNET	Reference architecture for autonomic SDN/NFV management (H2020 SELFNET)	Proposes multi-layer architecture (infrastructure, virtualized network, SON control, SON autonomic, NFV orchestration, etc.)	Autonomic SDN/NFV (5G networks)	– (architectural framework)
Al Salti *et al.* [10]	2023	EESLD	Event-driven link discovery framework for hybrid multi-controller SDN that improves discovery speed, efficiency, and scalability	Uses BFD to detect direct/indirect SDN links, sFlow for legacy link discovery, and a distributed messaging system to synchronize topology across controllers; implemented as an event-driven module integrated with SDN controllers	Hybrid multi-controller SDN (topology/link discovery, resilience)	Mininet emulator; RYU controller; sFlow server
Ferreira *et al.* [11]	2020	ArchSDN	Distributed multi-controller SDN with RL-based optimization	Partitions network into sectors, each managed by an ArchSDN controller using reinforcement learning to find near-optimal routes	Large-scale/IoT SDN networks	Simulated multi-controller SDN
Shirmarz *et al.* [12]	2020	IBM Autonomic SDN	Three-layer SDN controller using IBM autonomic model	Data, autonomic-control and route-learning layers with DNN traffic classification and greedy routing optimization	SDN QoS/performance management	POX controller (v0.2.0)
Arzo *et al.* [13]	2021	MANA-NMS	Multi-Agent autonomic SDN/NFV management with atomic decision agents	Theoretical modeling of SDN/NFV functions as agents; mathematical analysis of cooperative behavior	Autonomic SDN/NFV (e.g., 5G)	Mininet emulator
Min *et al.* [14]	2021	Self-adaptive Load Balancing	Multi-objective switch migration for load balancing in SDN-based IoT	Formulates switch migration as an optimization problem; applies heuristic algorithms (GA, SA, Greedy) within autonomic control loop	IoT-enabled SDN	Mininet (ARN topology, Topology Zoo); Ryu controller
Arzo *et al.* [15]	2022	Agentic 6G Architecture	Agent-based SDN/NFV architecture for 5G/B5G/6G networks	Conceptual integration of SDN, NFV, and edge into microservice/autonomic agent framework	5G/Beyond-5G (6G) networks	conceptual architecture
Araujo *et al.* [16]	2024	LLM-based Agentic SDN	Agentic SDN management framework leveraging large language models (LLMs)	Conceptual integration of LLM-based agents for adaptive SDN orchestration	SDN orchestration (LLM-based)	conceptual architecture

expected to help networks anticipate and predict stochastic disturbances, respond to them in real time, and adapt to prevent future disruptions [22].

Consequently, autonomic networking architectures are emerging as key paradigms for self-managing industrial networks. This transformation is crucial for complex and dynamic environments such as offshore WPPs, where harsh weather conditions, variable wind power outputs, and distributed control systems create continuous operational uncertainty [6], [23]. In such settings, networks that adapt, coordinate across multiple devices, and anticipate potential faults are essential for maintaining reliable and safe operation, minimizing downtime, and ensuring resilient performance without requiring constant human intervention.

There are two main reference autonomic network architectures that define the core components of autonomic behavior in future industrial networks [24], [25], [26]. The first architecture, introduced by the European Telecommunication Standards Institute (ETSI), is the *ETSI TS103195-2 General Autonomic Networking Architecture* (GANA) [25], which defines functional blocks for monitoring, decision-making, and enforcement, centered on a knowledge-plane control loop that spans multiple network planes. In comparison, IBM's autonomic computing model formalizes these ideas through the well-known MAPE-K loop [27], [28], which consists of Monitor, Analyze, Plan, and Execute functions operating over a shared knowledge base. These architectures implement a closed-loop control mechanism that enhances network performance and improves resource management functionalities [29].

By benchmarking these reference architectures, researchers continue to conduct empirical tests to evaluate their impact on network performance and resilience across different application domains, as summarized in Table 1. Early studies focused on policy-based automation. Bari et al. [7] developed PolicyCop, which was one of the first frameworks for autonomic SDN QoS management. It uses the SDN northbound API to automatically check and enforce policies, and experiments on an OpenFlow testbed showed it quickly reroute traffic and maintain performance. Similarly, Tsagkaris et al. [8] introduced AutoFlow, adding a policy layer to multi-domain SDN controllers. Tests on the GÉANT testbed demonstrated adaptive flow management across large networks. The next wave of research targeted cognitive and self-managing systems. SELFNET [9] proposed a multi-layer architecture for autonomic SDN/NFV management in 5G networks. ArchSDN [11] used reinforcement learning to optimize routing in large-scale networks, whereas IBM Autonomic SDN [12] applied deep learning for traffic classification and routing optimization. Min et al. [14]

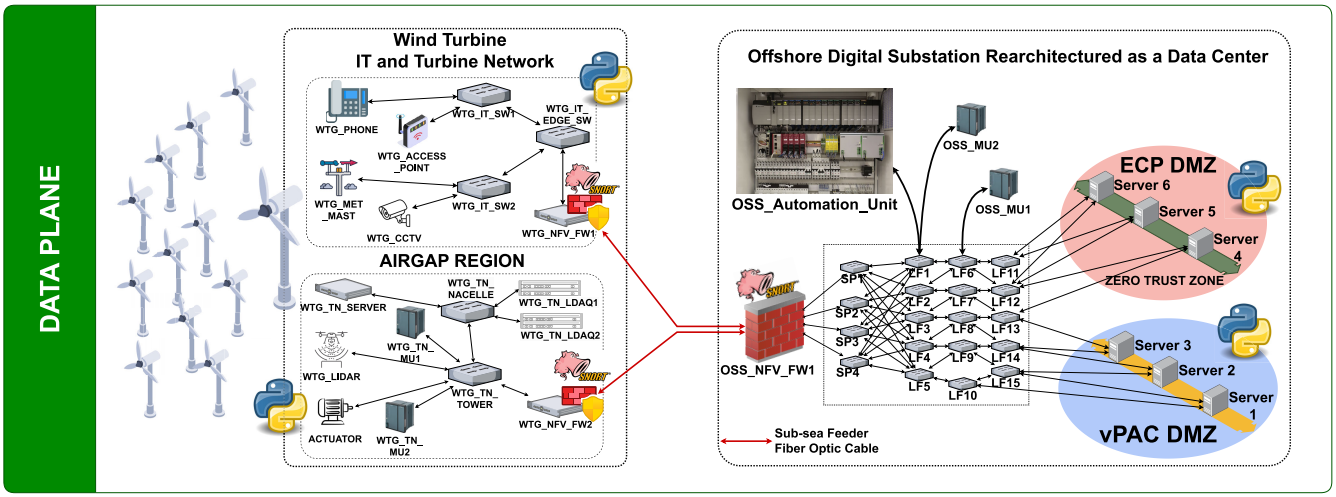


FIGURE 2. A software-defined industrial network data plane schematic showing fiber-optic interconnections between wind turbine generator nacelle and tower switches and the offshore hub's data center switches, which interface with the server cluster's physical network interfaces and virtualized environments within the ECP and vPAC server nodes.

focused on IoT networks, using heuristics to adaptively balance switch loads and improve the latency.

Recently, research has explored agent-based and cooperative systems. Arzo et al. [13] proposed MANA-NMS, which modeled SDN/NFV functions as independent agents, that adapt cooperatively to network changes. The Agentic 6G architecture [15] extended this idea to 5G/B5G networks by integrating SDN, NFV, and edge computing into autonomic microservice agents. LLM-driven orchestration is the latest stage of autonomy. Araujo et al. [16] demonstrated that large language models can act as intelligent agents in SDN, translating high-level user intents into policies and configurations to automatically manage the network. This progression shows how SDN autonomy has evolved: from simple policy enforcement, through cognitive control and cooperative agents, to AI-driven orchestration capable of interpreting high-level intents as conceptualized in Figure 1.

What is currently missing in the literature is not the existence of autonomic mechanisms themselves, but the ability to coordinate resilience actions across multiple failure domains within complex industrial SDN environments. Existing studies have made important contributions in areas such as policy enforcement [7], routing optimization [11], adaptive load balancing [14], and agent-oriented orchestration [15], [16]. However, most of these approaches address resilience from a largely isolated or domain-specific perspective, where individual autonomic functions operate independently to optimize a single network objective. As a result, limited attention has been given to how multiple intelligent agents can cooperate in real time to respond to concurrent disturbances affecting both the control and data planes of software-defined industrial networks.

This limitation becomes particularly significant in offshore wind power plant environments, where communication infrastructures operate under highly dynamic and uncertain conditions. Unlike conventional enterprise or generic SDN deployments, offshore industrial networks must simultaneously contend with deterministic traffic requirements, harsh environmental conditions, intermittent congestion, equipment thermal instability, and evolving cyber threats. In such environments, resilience cannot be treated as a standalone optimization problem confined to a single controller function or management layer. Instead, it requires coordinated, adaptive, and cross-plane decision-making mechanisms capable of maintaining operational continuity under compound failure scenarios.

To address these challenges, this paper proposes the OMARF framework, an orchestrated multi-agent resilience framework for software-defined industrial networks. In contrast to existing frameworks that primarily rely on centralized autonomic loops, isolated optimization strategies, or conceptual orchestration models, OMARF introduces a cooperative interaction model between specialized self-healing and self-defense agents operating across the network infrastructure. The framework enables the coordinated detection, analysis, and mitigation of control-plane failures, data-plane disruptions, congestion anomalies, Ethernet switch thermal instability, and cyber threats in near real time. Furthermore, the proposed architecture is specifically designed and empirically evaluated within the context of offshore wind power plant communication systems, where resilience, adaptability, and sustained service availability are operationally critical.

III. ORCHESTRATED MULTI-AGENT RESILIENCE FRAMEWORK (OMARF)

The proposed OMARF builds on these foundational models and extends their applicability to software-defined industrial networks deployed in offshore WPPs as illustrated in Figure 2. Its design follows the modular, feedback-driven principles of both GANA and MAPE-K while introducing

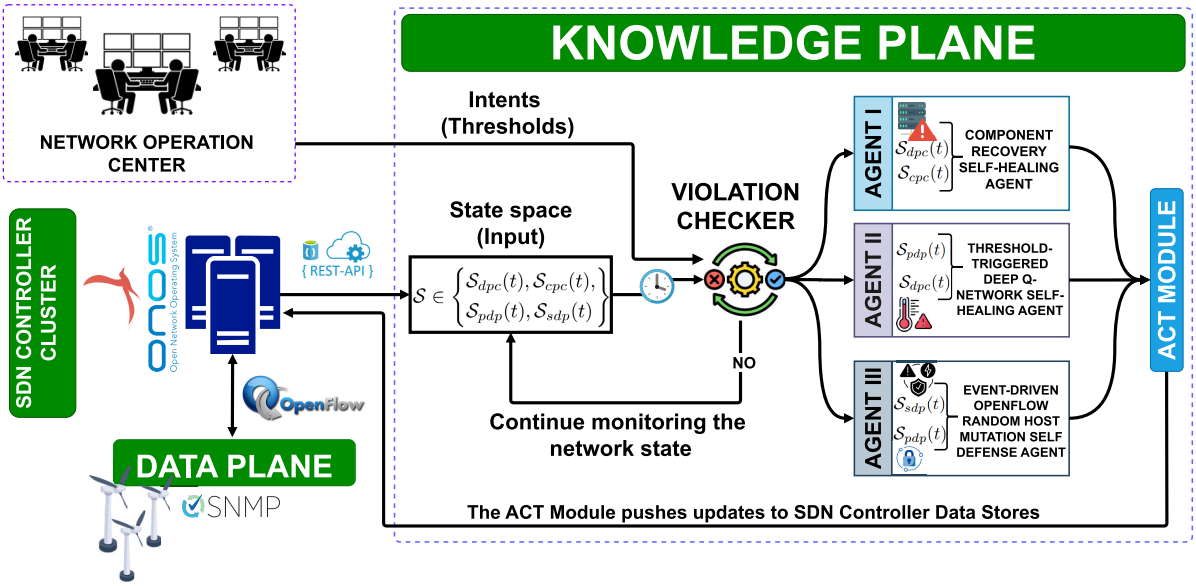


FIGURE 3. An orchestrated multi-agent resilience framework (OMARF) for software-defined industrial networks in offshore wind power plants.

additional capabilities, including fine-grained coordination, adaptive intelligence, integrated security response, and cloud-native scalability.

Figure 3 shows the schematic design of the proposed OMARF for an autonomic software-defined WPP industrial network [30]. The DATA PLANE consists of a WPP industrial switch network, which is centrally managed by an SDN controller (SDNC) cluster via a southbound interface protocol, such as OpenFlow. The CONTROL PLANE includes data stores such as the topology manager, flow rule manager, and statistics manager. It periodically polls the data plane switches to maintain these data stores. The KNOWLEDGE PLANE accesses the SDN controller's data stores through a northbound interface protocol (such as the RFC 2616 protocol used in RESTful APIs) to construct a dynamic STATE SPACE.

This STATE SPACE serves as an input to the VIOLATION CHECKER, which continuously evaluates the network's status against predefined thresholds. These thresholds are designed to enforce QoS/SLA standards and any custom performance requirements defined by the WPP operator. When a violation is detected, the violation checker determines which agent(s) should be activated to resolve the issue. In certain cases, multiple agents may be triggered simultaneously. The decisions made by the activated agents are compiled into a *.json* file by the ACT module and subsequently forwarded to the appropriate SDNC data store.

These are the main components of OMARF:

A. STATE SPACE (INPUT)

The state space ($\mathcal{S}$) comprises subsets containing time-based data on different features that determine the DATA PLANE and CONTROL PLANE status.

$$\mathcal{S} \in \{\mathcal{S}_{dpc}(t), \mathcal{S}_{pdp}(t), \mathcal{S}_{cpc}(t)\} \tag{1}$$

The DATA PLANE state space feature, $\mathcal{S}_{dp}(t) \in \{\mathcal{S}_{dpc}(t), \mathcal{S}_{pdp}(t)\}$, collects DATA PLANE specific component state parameters such as link status ($l_{(i,t)}$), node status ($\delta_{(k,t)}$), Ethernet switch thermal state ($\tau_{(k,t)}$), and Intrusion Detection System (IDS) nodes status ($\psi_{(j,t)}$) as denoted in equation 2.

$$\mathcal{S}_{dpc}(t) = \left\{ \begin{matrix} l_{(1,t)}, & l_{(2,t)}, & l_{(3,t)}, & \ldots, & l_{(i,t)}, \\ \delta_{(1,t)}, & \delta_{(2,t)}, & \delta_{(3,t)}, & \ldots, & \delta_{(k,t)}, \\ \tau_{(1,t)}, & \tau_{(2,t)}, & \tau_{(3,t)}, & \ldots, & \tau_{(k,t)}, \\ \psi_{(1,t)}, & \psi_{(2,t)}, & \psi_{(3,t)}, & \ldots, & \psi_{(j,t)}, \end{matrix} \right\} \tag{2}$$

where i is the number of active links, k is the number of active switches, j is the number of active IDS nodes at a given time t. DATA PLANE specific performance and security parameters such as link utilization ($\mu_{(i,t)}$), end-to-end path latency ($\lambda_{path_{(m,t)}}$), and IDS logs ($\psi_{logs_{(j,t)}}$) as shown in equation 3.

$$\mathcal{S}_{pdp}(t) = \left\{ \begin{matrix} \mu_{(1,t)}, & \mu_{(2,t)}, & \mu_{(3,t)}, & \ldots, & \mu_{(i,t)}, \\ \lambda_{(1,t)}, & \lambda_{path_{(2,t)}}, & \lambda_{(3,t)}, & \ldots, & \lambda_{(m,t)} \\ \psi_{logs_{(1,t)}}, & \psi_{logs_{(2,t)}}, & \ldots, & \psi_{logs_{(j,t)}} \end{matrix} \right\} \tag{3}$$

where m is the number of paths in the network at time t. The feature $\mathcal{S}_{cpc}(t)$ collects CONTROL PLANE specific component state parameters such as cluster status ($\xi_{(n,t)}$), controller CPU usage ($\rho_{(n,t)}$), controller memory usage ($\eta_{(n,t)}$), control loop delay ($\zeta_{(n,t)}$), and controller thread count

TABLE 2. State space variables and descriptions.

Variable	Description
$\mathcal{S}$	Overall state space
$\mathcal{S}_{dpc}(t)$	Data Plane Component states
$\mathcal{S}_{pdp}(t)$	Data Plane Performance states
$\mathcal{S}_{cpc}(t)$	Control Plane Component states
$l_{(i,t)}$	Status of link i at time t.
$\delta_{(k,t)}$	Status of node k at time t.
$\tau_{(k,t)}$	Thermal state of switch k at time t.
$\psi_{(j,t)}$	Status of IDS node j at time t.
$\mu_{(i,t)}$	Link utilization of link i at time t.
$\lambda_{path_{(m,t)}}$	End-to-end latency of path m at time t.
$\psi_{logs_{(j,t)}}$	IDS logs from node j at time t.
$\xi_{(n,t)}$	Status of controller cluster n at time t.
$\rho_{(n,t)}$	CPU usage of controller n at time t.
$\eta_{(n,t)}$	Memory usage of controller n at time t.
$\zeta_{(n,t)}$	Control loop delay of controller n at time t.
$\theta_{(n,t)}$	Thread count of controller n at time t.

($\theta_{(n,t)}$) as shown in equation 4.

$$\mathcal{S}_{cpc}(t) = \left\{ \begin{array}{c} \xi_{(1,t)},\ \xi_{(2,t)},\ \xi_{(3,t)},\ \ldots,\ \xi_{(n,t)}, \\ \rho_{(1,t)},\ \rho_{(2,t)},\ \rho_{(3,t)},\ \ldots,\ \rho_{(n,t)}, \\ \eta_{(1,t)},\ \eta_{(2,t)},\ \eta_{(3,t)},\ \ldots,\ \eta_{(n,t)}, \\ \zeta_{(1,t)},\ \zeta_{(2,t)},\ \zeta_{(3,t)},\ \ldots,\ \zeta_{(n,t)}, \\ \theta_{(1,t)},\ \theta_{(2,t)},\ \theta_{(3,t)},\ \ldots,\ \theta_{(n,t)} \end{array} \right\} \tag{4}$$

where n is the number of active controllers in the SDNC cluster at time, t.

The state space, $\mathcal{S}$, is periodically updated to ensure that the resilience framework operates on the most recent network status. Table 2 summarizes all the state space (input) elements. Additionally, a time-series database is implemented to store all state-space instances, enabling subsequent visualization, reporting, and improved decision-making. The agents provide more informed, context-aware responses based on these historical network states.

B. VIOLATION CHECKER

At the core of the proposed resilience framework lies the `VIOLATION CHECKER`, which ingests two critical inputs:

1) real-time state parameters derived from the dynamically updated state space, and
2) the predefined threshold values extracted from operator-defined intents and regulatory compliance constraints translated using LLM agents that interpret user intents into system intents (thresholds).

Using a policy engine to compare observed state, performance, and security parameters against the referenced intents, the `VIOLATION CHECKER` detects deviations, policy breaches, or potential service degradations [31]. This enables the timely triggering of the appropriate agent. For the proposed WPP network, $\mathcal{S}_{intents}$ in equation 5 denotes the predefined policies and thresholds used in the `VIOLATION CHECKER` based on the stipulated QoS/SLA standards.

$$\mathcal{S}_{intents} = \left\{ \begin{array}{l} \mu_{(i,t)} \leq 80\%, \\ \rho_{(n,t)} \leq 75\%, \\ \lambda_{path_{(m,t)}} < 3\,\text{ms}, \\ \eta_{(n,t)} \leq 70\%, \\ \zeta_{(n,t)} \leq 10\,\text{ns}, \\ \tau_{(k,t)} \in [18°\text{C}, 27°\text{C}], \\ \theta_{(n,t)} \leq 200, \\ (up) \in \{\delta_{(k,t)}, l_{(i,t)}, \psi_{(j,t)}\} \end{array} \right. \tag{5}$$

Additionally, the node status in both the `CONTROL PLANE` and `DATA PLANE` is expected to be `UP/ACTIVE` to ensure operational continuity and coordination across the network. The policy engine embedded within the `VIOLATION CHECKER` continuously evaluates state space input parameters against a set of predefined rules using a rule engine, such as the `Open Policy Agent` using *Rego*, or a structured decision model, such as a *decision tree*. Detected violations are then passed to a mapping layer, where specific combinations of anomalies are analyzed to accurately determine and trigger the most appropriate agent(s) for response.

C. DECISION-MAKING AGENTS

When `AGENT I`, a *"Component Recovery Self-Healing Agent"* [30], is activated, it polls the status data of all the relevant components:

$$\{\xi_{(n,t)}, \zeta_{(n,t)}, \rho_{(n,t)}, \theta_{(n,t)}, \delta_{(k,t)}, \psi_{(j,t)}, l_{(i,t)}\}$$

from both the `CONTROL PLANE` and `DATA PLANE` to respond to component failure. Then, it applies a *decision tree* algorithm to determine the most appropriate mitigation or adaptation strategy.

At the `CONTROL PLANE`, when the controller's CPU or memory usage exceeds a predefined threshold, the `ACT` module triggers the Kubernetes `Horizontal Pod Autoscaler`, supporting the SDN controller cluster to reallocate resources dynamically. In critical situations, it may also initiate a virtual machine (VM) migration to redistribute the load, thereby ensuring performance consistency and preventing controller saturation. When the control loop delays in the SDNC cluster exceed the allowable threshold and cause misalignment or loss of synchronization between controller nodes, the `ACT` module actively resynchronizes the SDNC. This involves prioritizing high-latency flows for offloading, adjusting the control-loop interval, or reallocating flow rules to other controller nodes to restore the cluster coherence.

Finally, when the thread count of an individual SDNC node surpasses its threshold, the `ACT` module spawns additional controller threads or instances (depending on the deployment model). It redistributes tasks using Kubernetes `ReplicaSets` or service-mesh-level load balancing to maintain responsiveness and prevent service degradation. At the `DATA PLANE`, when IDS nodes, switch nodes, or network links become inactive or unresponsive, the `ACT`

module initiates a series of coordinated resilience actions. Specifically, it triggers controlled switch reboots to recover from transient faults, notifies network engineers of exact link failures, and proactively recalculates alternative paths using preconfigured failover or dynamic traffic engineering policies. This rerouting mechanism leverages real-time topology updates to divert traffic away from the affected segments, thereby minimizing packet loss and preserving network continuity.

When AGENT II, a *"Threshold-Triggered Deep Q-Network Self-Healing Agent"* (TTDQSHA) [32], is activated, it collects comprehensive network behavior and performance status data: $\{\mu_{(i,t)}, l_{(i,t)}, \delta_{(k,t)}, \lambda_{\text{path}_{(m,t)}}, \tau_{(k,t)}\}$. These data are processed with a pre-trained and validated TTDQSHA model to determine the optimal path for rerouting traffic away from malfunctioning switches that exhibit thermal instability. AGENT II operates in conjunction with AGENT I, as accurate node health information is crucial before applying traffic engineering techniques to reallocate resources. During its training phase, AGENT II learns optimal decision-making policies using a well-defined reward function that captures diverse network behavior scenarios and performance goals.

Once deployed, the agent rapidly responds to real-time network anomalies, providing significantly faster remediation than baseline routing protocols such as OSPF, which rely on the Dijkstra algorithm and lack adaptive learning mechanisms. Finally, AGENT II pushes flow control instructions in *.json* format to the ACT module. These instructions are subsequently parsed and inserted into the SDN controller's flow rule manager, facilitating dynamic enforcement of the updated routing policies.

When AGENT III, an *"Event-driven OpenFlow Random Host Mutation (*OF−RHM*) Self-Defense Agent"* [33], is activated, it collects comprehensive network performance and security status data:

$$\{\mu_{(i,t)}, l_{(i,t)}, \delta_{(k,t)}, \lambda_{\text{path}_{(m,t)}}, \psi_{(j,t)}, \psi_{\text{logs}_{(j,t)}}\}$$

This data is analyzed to identify potential indicators of compromise and latent attack signatures across both early-stage reconnaissance and late-stage exploitation phases. The agent leverages event-driven triggers to dynamically initiate host mutation processes, periodically altering host identifiers and flow rules via OpenFlow, thereby obscuring the network topology and reducing attacker persistence. AGENT III operates in coordination with AGENT I, ensuring that mutation strategies are only applied when all node health indicators confirm a stable network state.

During its design and simulation phase, AGENT III is evaluated using representative attack vectors and response metrics to calibrate its mutation frequency, scope, and effectiveness in degrading the accuracy of adversarial reconnaissance. Once deployed, the agent autonomously enhances cyber resilience by implementing network-level Moving Target Defense techniques that proactively disrupt adversarial planning cycles. Compared to static defense mechanisms or signature-based IDS frameworks, AGENT III offers adaptive, topology-aware obfuscation with minimal disruption to legitimate traffic flows. Finally, AGENT III transmits mutation policies and updated flow rules in *.json* format to the ACT module, which inserts them into the SDNC's flow rule manager.

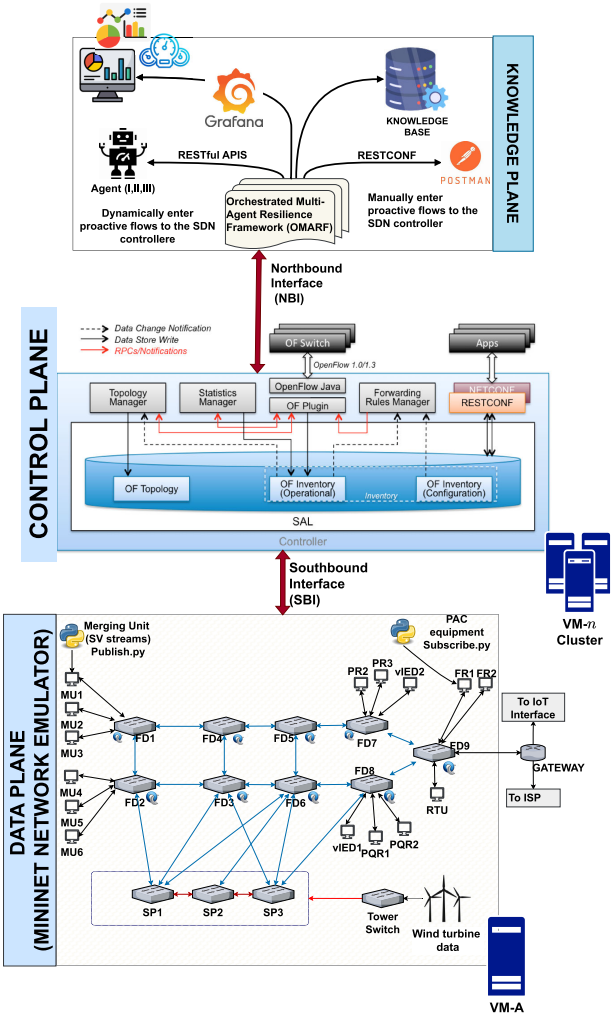


FIGURE 4. Experimental SURF HPC cloud-based testbed for emulating the software-defined industrial network, comprising a Mininet VM for the data plane, an ONOS SDN controller cluster for the control plane, and a Linux-based knowledge plane VM interacting via RESTful APIs.

IV. METHODOLOGY

A. EXPERIMENTAL SETTING

The experiment is conducted on a SURF HPC cloud-based testbed comprising: a Mininet VM (*version 2.3.0*) that emulates the WPP's data plane switch network, an ONOS SDNC cluster VMs (*version 2.7.0 Snowy Owl*) at the control plane, and a Linux-based knowledge plane VM that interacts with the SDNC cluster through the Northbound Interface using RESTful APIs. Each VM is provisioned with 8 vCPUs,

TABLE 3. Offshore wind power plant data communication parameters [6].

Service	Communication Direction	Priority	Data rate
Protection traffic	WTG → vPAC	1	76,816 bytes/s
Analogue measurements	WTG → vPAC/ECP	2	225,544 bytes/s
Status information	WTG → ECP	2	58 bytes/s
Reporting and logging	WTG → ECP	3	15 KB every 10 minutes
Video surveillance	WTG → ECP	4	250 kb/s – 1.5 Mb/s
Control traffic	vPAC → WTG	1	20 kb/s per turbine
Data polling	ECP/vPAC → WTG	2	2 KB every second
Internet connection	Internet → WTG/ECP/vPAC	3	1 GB every two months

TABLE 4. Omarf agent activation per scenario.

Scenario	Scenario Description	I	II	III
1	Controller CPU overload ($\rho_{(n,t)} > 75\%$) and switch overheating ($\tau_{(k,t)} > 27°C$)	✓	✓	✗
2	Link failures ($l_{(i,t)} = \texttt{DOWN}$) and IDS anomalies ($\psi_{\log s_{(j,t)}}$)	✓	✗	✓
3	Thread overload ($\theta_{(n,t)} > 200$) and IDS intrusion	✓	✗	✓
4	Thermal faults, IDS alerts, and link congestion ($\mu_{(i,t)} > 80\%$)	✗	✓	✓
5	Full-stack degradation event, where $\mathcal{S}_{intents}$ is completely violated	✓	✓	✓

32 GB RAM, and 100 GB SSD storage, running Ubuntu 22.04 LTS.

1) AT THE DATA PLANE

The proposed software-defined industrial network was emulated in Mininet as illustrated in Figure 4. Custom scripts were run on the host machines to simulate real wind power plant data transfer [6] at the data rate denoted in Table 3. Some hosts were configured as SNORT-based Intrusion Detection System (IDS) nodes to monitor and analyze the security state of network nodes positioned at strategic locations.

A publish–subscribe communication model was adopted for managing data traffic. In this setup, sensors and local data acquisition units published their data to the switch network and MQTT brokers, while the edge computing platform (ECP) nodes subscribed to these data streams through designated topics. The ECP nodes also published response data back to the broker, to which the actuators subscribed to perform specialized control actions within the wind turbine system. Meanwhile, the wind turbine–based and offshore digital substation–based merging units (MUs) published Sample Values (SVs) to the switch network, with the virtual protection, automation, and control (vPAC) nodes periodically subscribing to these data samples as captured from the data plane. Furthermore, the IEDs published Generic Object-Oriented Substation Event (GOOSE) messages, which the actuators subscribed to for real-time triggering of protection and control operations. The traffic flows were monitored using ONOS's GUI and packet-tracing tools as illustrated in Figure 5.

2) AT THE CONTROL PLANE

A cluster of three ONOS SDN controllers was deployed using the `org.onosproject.cluster-ha` feature, with the Raft consensus algorithm ensuring leader election and state consistency across nodes. Southbound OpenFlow messages between the controllers and switches were monitored and captured for performance analysis, as illustrated in Figure 5. The SDNC cluster interfaced with the knowledge plane via a RESTful northbound API for information exchange and coordinated control actions. In addition, ONOS provides distributed data stores and state-management primitives through its `DistributedStore` framework, maintaining consistent network-state information, including device topology, host mappings, flow rules, and intent states, across the cluster. These are managed through core services including the `DeviceService`, `LinkService`, `HostService`, `FlowRuleService`, and `StatisticsService`, which collectively abstract and aggregate the dynamic behavior of the data plane. Through these services, the ONOS SDN cluster continuously collects telemetry data, including packet-in events, flow statistics, port utilization metrics, link latency, and device reachability.

3) AT THE KNOWLEDGE PLANE

The proposed `OMARF` decision-making agents, as described in Section III-C, were executed within the knowledge plane. Furthermore, the testbed integrates Prometheus and Grafana for real-time monitoring and visualization of network status, performance indicators, security metrics, and controller health. Prometheus continuously scrapes metrics exposed by the ONOS SDNC data stores and network devices through RESTful endpoints, storing time-series data for analysis and alert generation, while Grafana provides interactive dashboards that visualize these metrics, providing for the `OMARF` agents to correlate controller state, data plane performance, and resilience indicators in real time for adaptive decision-making.

B. OMARF PERFORMANCE EVALUATION UNDER MULTI-FAULT SCENARIOS

To evaluate the performance of the proposed `OMARF`, this study defines five test case scenarios representing realistic multi-fault and multi-metric degradation events across both the `DATA PLANE` and `CONTROL PLANE`. These scenarios were selected to assess how `OMARF` coordinates, responds, and scales under concurrent violations of service-level thresholds and operator-defined intents, as outlined in equation 5. Each scenario, summarized in Table 4, was evaluated on the SURF HPC testbed and compared against a baseline configuration where none of the state-space input parameters violated predefined thresholds or intents.

Faults were systematically injected through a combination of Linux commands, ONOS REST APIs, and Python-based event triggers to assess the agents' adaptive behaviors. CPU stress conditions on the SDN controllers and switches were

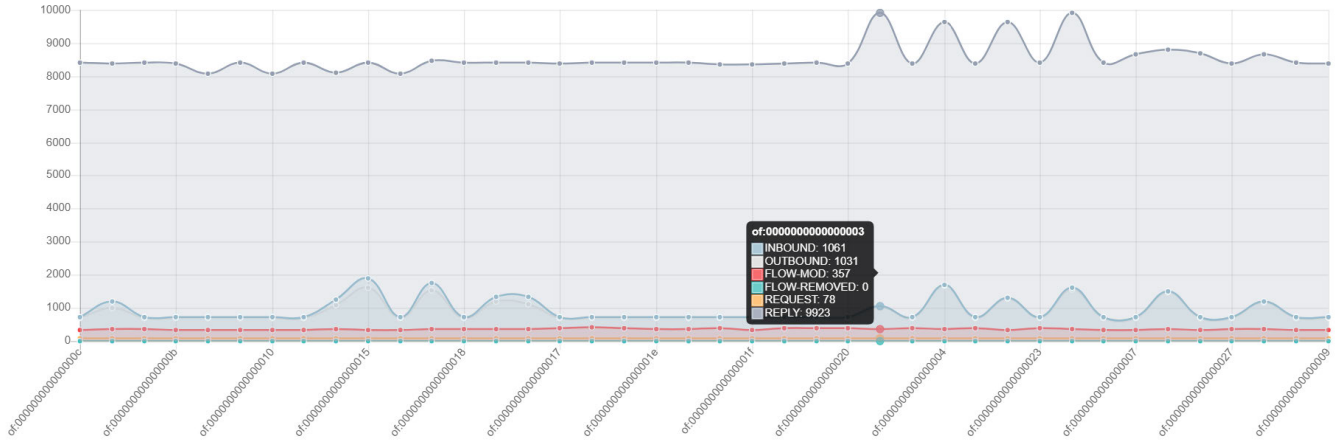


FIGURE 5. Monitoring and capturing of southbound OpenFlow messages between ONOS controllers and switches for performance analysis.

generated using `stress-ng` and `cpulimit`, gradually driving utilization beyond 75% to activate Agent I's response mechanisms. Thermal variations were reproduced using a custom temperature script that retrieved switch utilization data from the `KNOWLEDGE BASE` and incorporated ambient environmental factors. By adjusting the actual air-conditioning values in the testbed, the script generated synthetic sensor readings (via `SNMP` and `MQTT` feeds) exceeding 27°C, thereby triggering the thermal management logic of Agent II.

Network degradation was introduced through Mininet's `link` command, which selectively disabled or throttled links, while controlled `iperf3` traffic generation induced congestion levels surpassing 80% utilization. Control-plane threading stress was simulated by increasing ONOS intent installation requests and concurrent flow-rule updates via the REST API, leading to elevated $\theta_{(n,t)}$ metrics. To emulate security disturbances, SNORT IDS nodes deployed at strategic network points produced anomaly logs and intrusion alerts based on rule-driven synthetic traffic patterns (e.g., port scans and malformed packets).

Finally, scalability tests were conducted by incrementally increasing the number of OpenFlow switches ($N = 46, 64, 86, 120$) and the corresponding flow rules managed per controller. The clustered ONOS controllers, operating under the Raft consensus algorithm, maintained state consistency, while key performance metrics, including decision latency, agent activation time, and packet loss rate, were measured under each network scale.

C. OMARF SYSTEM HEALTH SCORE $\mathcal{H}(T)$ AND RISK SCORE $\mathcal{R}(T)$

The resilience factor of the proposed `OMARF` was modeled as a discrete-time dynamical system. This formulation enables modeling `OMARF`'s behavior over extended time intervals, capturing how agent decisions shape system trajectories under dynamic, uncertain conditions. The resulting metrics were displayed on a Grafana dashboard as illustrated in Figure 6. The Health Score $\mathcal{H}(t)$ integrates normalized key performance indicators (such as *control-plane convergence time, data-plane throughput, packet-loss rate, and synchronization fidelity*) to assess the operational state of the network. It is given as a function of multiple data-plane and control-plane system state parameters as denoted in equation 6,

$$\mathcal{H}(t) = \frac{1}{\mathcal{N}} \times \sum_{(i=1)}^{\mathcal{N}} \omega_i \left(\frac{\mathcal{P}_i(t)}{\mathcal{P}_i^{max}} \right), \mathcal{P}_i(t) \in \mathcal{S}(t) \tag{6}$$

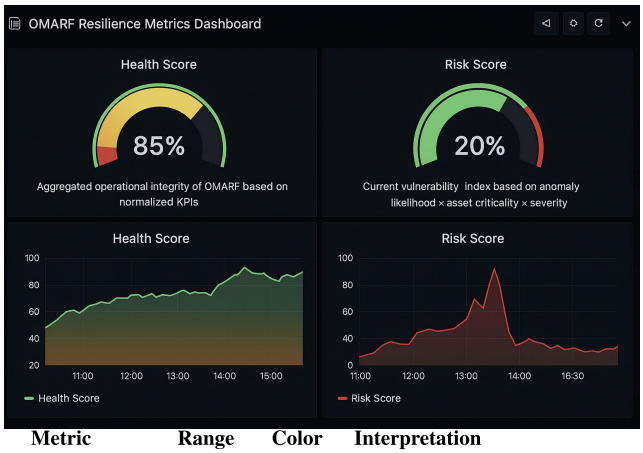
where $\mathcal{P}_i(t)$ is the observed parameter value at time t and $\mathcal{P}_i^{max}$ represents the maximum acceptable threshold for each parameter, ω_i is the weight assigned to each observed parameter with path latency, switch, and IDS node status being weighted higher than the other observed parameters, and $\mathcal{N}$ being the total number of parameters observed [34].

In contrast, the Risk Score $\mathcal{R}(t)$ characterizes the current risks by synthesizing the likelihood of detecting anomalies (*such as link failures, switch overheating, or intrusion patterns*) with asset criticality and known vulnerability severities from the NIST threat vulnerability databases. It quantifies the potential for service disruption, security breach, or performance degradation using a time-weighted violation score as denoted in equation 7,

$$\mathcal{R}(t) = \frac{1}{\mathcal{T}} \sum_{\beta=0}^{\mathcal{T}} \alpha^{(t-\beta)}.\mathcal{V}(\mathcal{S}_\beta, \mathcal{S}_{threshold}) \tag{7}$$

where $\mathcal{T}$ is the size fo the historical time window (e.g. 5 minutes), β is the index over past time steps $[0, \mathcal{T})$ and $\alpha^{(t-\beta)}$ is the time-decay factor computed at a given time t, $\mathcal{V}(\mathcal{S}_\beta, \mathcal{S}_{threshold})$ is the violation function that computes from the `VIOLATION CHECKER` how many violations have occurred at a time β [35]. The result is a time series list of state snapshots which can be used to compute the resilience index, $\mathcal{RI}(t)$ as denoted in equation 8,

$$\mathcal{RI}(t) = \omega_{\mathcal{H}}\mathcal{H}(t) + \omega_{\mathcal{R}}(1 - \mathcal{R}(t)) \tag{8}$$



Metric	Range	Color	Interpretation
Health Score	0–60		Poor operational integrity
	60–90		Degraded but functional
	90–100		Stable and healthy
Risk Score	0–30		Low risk, system safe
	30–50		Moderate risk, monitor
	50–100		High risk, imminent disruption

FIGURE 6. OMARF Resilience Metrics Grafana dashboard showing real-time Health and Risk Scores. The dashboard visualizes metrics collected by Prometheus, which queries and processes data extracted from the knowledge base.

where $\omega_{\mathcal{H}}$ and $\omega_{\mathcal{R}}$ are weighted factors that are tuned to the operators' priorities [36].

Together, these scores provide a comprehensive, continuously updated assessment of OMARF's resilience, as well as the network's performance and behavioral stability in managing WPP operations under dynamic, uncertain conditions.

V. EMPIRICAL FINDINGS AND ANALYSIS

The performance of the proposed OMARF framework was evaluated using the experimental testbed described in Section IV-A. The OMARF agents were triggered in response to events detected by the VIOLATION CHECKER. This section presents two main results: (i) the behavior and performance of the network under different conditions, and (ii) the corresponding resilience scores and scalability analysis.

A. NETWORK PERFORMANCE AND FAULT RESPONSE

Figure 7 presents the network throughput response under sequential fault scenarios. During the initial fault stages (scenarios I–III, 10:47:00–10:49:30), the network experienced controller CPU overload, thermal violations, and link-level anomalies. OMARF maintained stable throughput with only a minor increase in packet loss (≈1.3%) as observed on the *iperf3* logs. This demonstrated the framework's ability to *localize micro-faults* before they escalated, primarily through Agent I's rapid component-level mitigation and Agent II's dynamic congestion control. End-to-end latency remained below 2.5 ms, and jitter remained under 2.15 ms, confirming that QoS requirements were preserved during these disruptions.

In the composite disruption phase (Scenarios IV–V), concurrent IDS alerts, network traffic congestion, and full-stack threshold violations activated all three agents, shifting the system into a high-alert state ($\mathcal{S}_{health}$ = Busy). A transient dip in throughput occurred, falling briefly below 2,500 packets/s, but the system re-stabilized within 30 seconds, achieving a packet delivery ratio of at least 98.7%. The activation latency across agents remained under 5 seconds, highlighting OMARF's rapid response capability. Compared with a baseline SDN setup without OMARF, throughput recovery improved by approximately 23%, while mean recovery time decreased by 42% as observed in the *iperf3* logs. These results indicate that *multi-agent collaboration*, rather than isolated reactions, is essential for sustaining end-to-end QoS in multi-fault industrial network environments. The analysis demonstrates that OMARF's orchestrated multi-agent approach effectively mitigates cascading failures, maintains resilience, and preserves operational performance under realistic fault conditions.

The scalability of the OMARF framework was evaluated by varying the number of data-plane switches (N = 46, 64, 86, 120). Figure 8 presents the packet loss rate (PLR) observed across these network sizes for the five operational scenarios summarized in Table 4. Overall, the results indicate that multi-agent coordination within OMARF scales effectively, maintaining low packet loss and stable activation patterns as network size increases. Notably, Scenario 5 exhibits a higher PLR due to full-stack degradation, while intermediate scenarios (particularly Scenario 3) reflect localized stochastic disruptions in the data plane. These variations highlight that, although scalability remains predictable, system stress at larger scales can amplify transient disruptions linked to controller and IDS activity.

B. AGENT OVERHEAD AND COMPUTATIONAL EFFICIENCY

Further, the CPU overhead introduced by OMARF's agents across five fault scenarios is illustrated in Figure 9. Agent I generates the baseline monitoring load, while Agents II and III incrementally increase utilization as the network experiences more complex disruptions. In Scenario 1, total CPU usage remained approximately 55.6%, rising to 78.7% when all agents were active in Scenario 5. The growth in overhead indicates predictable scalability and confirms the framework's suitability for real-time deployment in edge and industrial environments. Despite the increase in compound faults, CPU utilization remained well below critical thresholds, demonstrating that OMARF performs continuous monitoring, congestion control, and cyber defense without overloading the knowledge plane. This efficiency ensures that multi-agent orchestration enhances resilience without introducing performance bottlenecks.

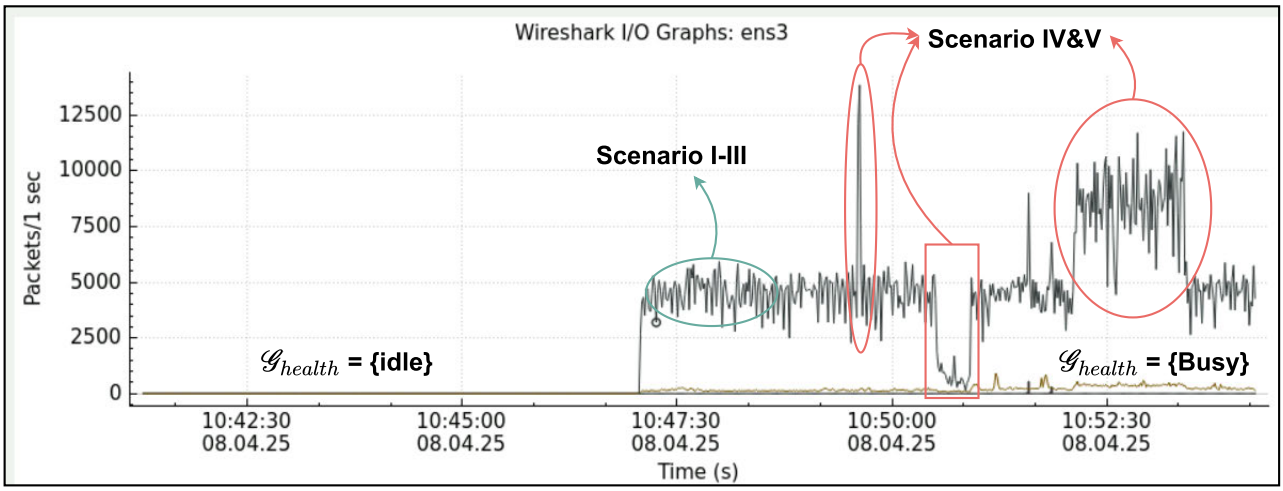


FIGURE 7. Network throughput behavior (packets/sec) observed on interface `ens3` during sequential activation of OMARF agents across five fault scenarios.

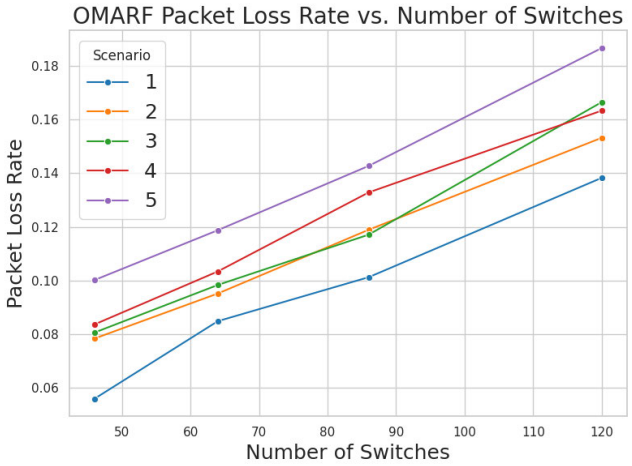


FIGURE 8. Packet Drop Rate on OMARF implemented in testbed with varying number of switches ($N = 46, 64, 86, 120$) at the data plane.

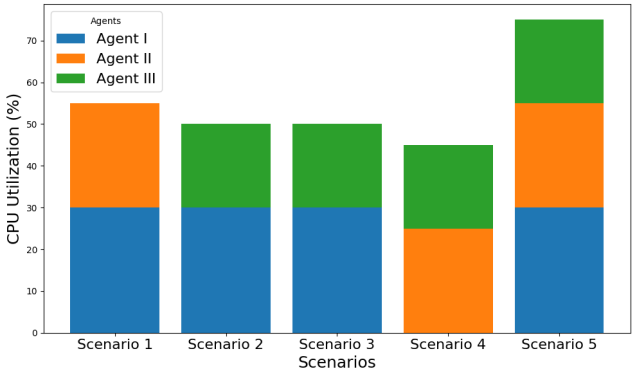


FIGURE 9. Stacked bar chart showing the cumulative CPU overhead (%) introduced by Agents I, II, and III under five different fault-injection scenarios.

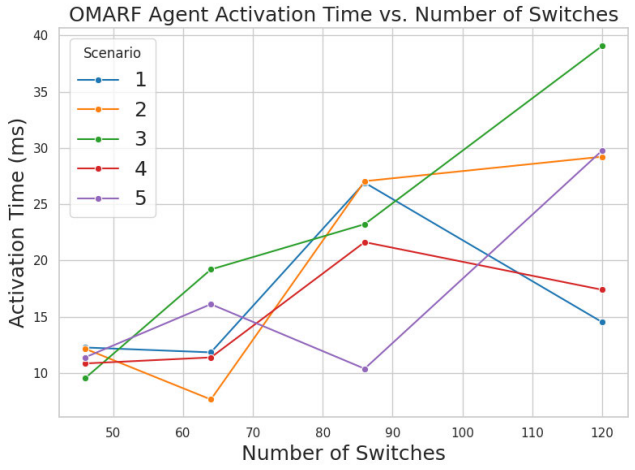
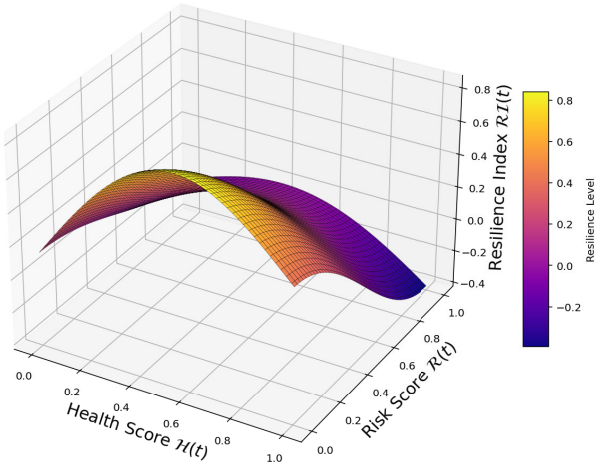


FIGURE 10. Agent activation process, illustrating the computation of activation time across network sizes.

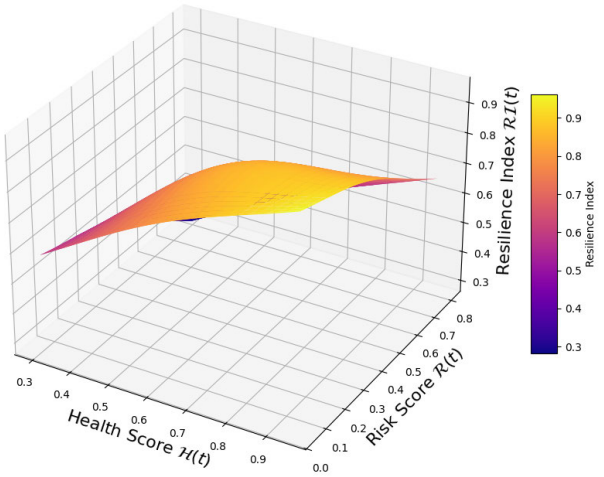
C. RESILIENCE INDEX AND ADAPTIVE RECOVERY

Furthermore, experiments conducted across varying data-plane network sizes measured the agent activation time, defined as the interval from the *OBSERVE* module's state-space generation to the execution of the *VIOLATION CHECKER* and the subsequent mapping of detected violations to specific agents. Figure 10 depicts temporal agent activation patterns under escalating fault scenarios, showing how the framework coordinates responses as disruptions intensify. Notably, Scenario 3 exhibits higher activation times, primarily due to increased SDN controller cluster thread load and intrusions that require external queries to the NIST threat database. This shows that the control plane is arguably the most critical in the architecture and must be consistently monitored.

To quantify OMARF's impact on network robustness, Health Scores $\mathcal{H}(t)$ and Risk Scores $\mathcal{R}(t)$ were computed from time-series port statistics, yielding the Resilience Index $\mathcal{RI}(t)$. Figures 11(a) and (b) show 3D resilience surfaces before and after OMARF activation. Before OMARF



(a) Resilience surface before OMARF intervention, showing lower resilience levels under deteriorating health and increasing risk conditions.



(b) Resilience surface after OMARF activation, showing an upward bulge where resilience improves as health is restored and risk mitigated.

FIGURE 11. Comparison of Resilience Index $\mathcal{RI}(t)$ before and after OMARF intervention. The post-intervention plot demonstrates the system's recovery capabilities and improved robustness under coordinated agent actions.

intervention, the resilience surface declines steadily as health deteriorates or risk accumulates, highlighting the system's vulnerability to cascading failures. After activation, the surface exhibits a pronounced upward curvature, or *"resilience bulge,"* corresponding to restored operational health and mitigated risk. Post-activation, the average resilience index increased from 0.63 to 0.91, representing a 44% improvement. This recovery is attributable to the coordinated actions of all three agents, including controller autoscaling, reinforcement-learning-based path reconfiguration, and proactive cyber-defense strategies. The steeper gradient along the risk axis reflects the applied weighting factors ($\omega_{\mathcal{H}} = 0.6785$, $\omega_{\mathcal{R}} = 0.3215$), which prioritize operational continuity while still mitigating system risks. These results demonstrate that OMARF's multi-agent orchestration actively restores resilience in real time, enabling the network to maintain stability under complex, compound disruptions.

D. DISCUSSION

The experimental results demonstrate that OMARF achieves adaptive, self-managing resilience in software-defined industrial networks through coordinated multi-agent intelligence. The framework maintains high throughput, low latency, and a packet delivery ratio above 98% under both single-fault and compound-fault scenarios. Resilience, measured by $\mathcal{RI}(t)$, improved by 44% relative to baseline SDN networks, while CPU overhead remained below 17% and activation latency under 5 s. These findings indicate that OMARF's multi-agent orchestration, rather than isolated agent action, is key to sustaining operational performance during cascading disruptions. The highlighted design principles include rapid fault isolation, proactive congestion control, and integrated cybersecurity monitoring. However, deployment in legacy networks remains challenging due to heterogeneous telemetry interfaces and limited SDN interoperability. Future work will investigate hybrid controller designs, protocol translation layers, and phased agent integration to extend OMARF to mixed SDN–non-SDN environments, enabling gradual adoption in industrial settings without disrupting existing infrastructure.

VI. CONCLUSION

This paper presented the Orchestrated Multi-Agent Resilience Framework (OMARF) for autonomic software-defined industrial networks. The framework coordinates self-healing and self-defense agents across the control, data, and knowledge planes to handle failures, thermal issues, control-plane overload, and ongoing cyber threats. Unlike traditional approaches that treat resilience and security as separate problems, OMARF brings them together through a unified orchestration layer that supports real-time response to changing network conditions.

Simulation results show that OMARF improves overall network performance and stability, achieving up to 28.3% lower path latency while maintaining higher throughput under a range of fault and attack scenarios. Even under degraded conditions, the framework keeps the resilience index above 0.625, showing consistent and stable behavior when the network is under stress. These results highlight the value of coordinated, policy-driven decision-making for maintaining reliable industrial network operations.

Future work will focus on testing OMARF in larger and more distributed environments, including scenarios with multiple controllers and more complex network topologies. It will also explore the use of digital twins for predictive analysis and safer evaluation of resilience actions, alongside improved modeling of thermal dynamics and failure behavior, and enhanced defenses against coordinated

cyber-physical attacks. Also, interoperability with legacy and heterogeneous industrial systems will be investigated to support practical deployment in real-world settings.

ACKNOWLEDGMENT

The authors gratefully acknowledge the contributions and collaborations that made this study possible. In particular, they thank EDF Research and Development Smart Grid Laboratories, Paris-Saclay, France, for their invaluable support in the design, validation, and testing of Agent I; Ørsted System Design Unit, Gentofte, Denmark, and Warsaw, Poland, for their contributions to the development and validation of Agent II; and TU Delft's Control Room of the Future, Delft, The Netherlands, for their contribution to the design and implementation of Agent III.

REFERENCES

[1] S. M. Alizadeh and C. Ozansoy, "The role of communications and standardization in wind power applications—A review," *Renew. Sustain. Energy Rev.*, vol. 54, pp. 944–958, Feb. 2016.

[2] A. Mwangi, E. Fumagalli, M. Gryning, and M. Gibescu, "Building resilience for SDN-enabled IoT networks in offshore renewable energy supply," in *Proc. IEEE 9th World Forum Internet Things (WF-IoT)*, Oct. 2023, pp. 1–2.

[3] A. Al Abdulwahid, "Cyber-physical threat mitigation in wind energy systems: A novel secure architecture for industry 4.0 power grids," *Energy Informat.*, vol. 7, no. 1, p. 141, Dec. 2024.

[4] A. A. Ganin, E. Massaro, A. Gutfraind, N. Steen, J. M. Keisler, A. Kott, R. Mangoubi, and I. Linkov, "Operational resilience: Concepts, design and analysis," *Sci. Rep.*, vol. 6, no. 1, pp. 1–12, Jan. 2016.

[5] B. J. van Asten, N. L. M. van Adrichem, and F. A. Kuipers, "Scalability and resilience of software-defined networking: An overview," 2014, *arXiv:1408.6760*.

[6] A. Mwangi, R. Sahay, E. Fumagalli, M. Gryning, and M. Gibescu, "Towards a software-defined industrial IoT-edge network for next-generation offshore wind farms: State of the art, resilience, and self-X network and service management," *Energies*, vol. 17, no. 12, p. 2897, Jun. 2024.

[7] M. F. Bari, S. R. Chowdhury, R. Ahmed, and R. Boutaba, "PolicyCop: An autonomic QoS policy enforcement framework for software defined networks," in *Proc. IEEE SDN Future Netw. Services (SDN4FNS)*, Nov. 2013, pp. 1–7.

[8] K. Tsagkaris, M. Logothetis, V. Foteinos, G. Poulios, M. Michaloliakos, and P. Demestichas, "Customizable autonomic network management: Integrating autonomic network management and software-defined networking," *IEEE Veh. Technol. Mag.*, vol. 10, no. 1, pp. 61–68, Mar. 2015.

[9] P. Neves, R. Calé, M. R. Costa, C. Parada, B. Parreira, J. Alcaraz-Calero, Q. Wang, J. Nightingale, E. Chirivella-Perez, W. Jiang, H. D. Schotten, K. Koutsopoulos, A. Gavras, and M. J. Barros, "The SELFNET approach for autonomic management in an NFV/SDN networking paradigm," *Int. J. Distrib. Sensor Netw.*, vol. 12, no. 2, Feb. 2016, Art. no. 2897479.

[10] I. A. Salti and N. Zhang, "An effective, efficient and scalable link discovery (EESLD) framework for hybrid multi-controller SDN networks," *IEEE Access*, vol. 11, pp. 140660–140686, 2023.

[11] C. Ferreira, S. Sargento, and A. Oliveira, "ArchSDN: A reinforcement learning-based autonomous OpenFlow controller with distributed management properties," *Social Netw. Appl. Sci.*, vol. 2, no. 9, p. 1533, Sep. 2020.

[12] A. Shirmarz and A. Ghaffari, "An autonomic software defined network (SDN) architecture with performance improvement considering," *J. Inf. Syst. Telecommun. (JIST)*, vol. 8, no. 30, pp. 121–129, Aug. 2020.

[13] S. T. Arzo, R. Bassoli, F. Granelli, and F. H. P. Fitzek, "Multi-agent based autonomic network management architecture," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 3, pp. 3595–3618, Sep. 2021.

[14] Z. Min, H. Sun, S. Bao, A. S. Gokhale, and S. S. Gokhale, "A self-adaptive load balancing approach for software-defined networks in IoT," in *Proc. IEEE Int. Conf. Autonomic Comput. Self-Organizing Syst. (ACSOS)*, Sep. 2021, pp. 11–20.

[15] S. T. Arzo, D. Scotece, R. Bassoli, F. Granelli, L. Foschini, and F. H. P. Fitzek, "A new agent-based intelligent network architecture," *IEEE Commun. Standards Mag.*, vol. 6, no. 4, pp. 74–79, 2022, doi: 10.1109/MCOMSTD.0001.2100053.

[16] A. S. Araujo, J. M. O. D. Mercês, R. L. Da Silva, A. V. De Alencar, I. F. Passos, M. P. Sousa, M. C. Dias, T. F. Meneses, and D. F. Santos, "An agentic approach for dynamic software-defined network management using large language models," in *Proc. IEEE Conf. Netw. Function Virtualization Softw. Defined Netw.*, Apr. 2024, pp. 221–226.

[17] P. Vizarreta, A. V. Bemten, E. Sakic, K. Abbasi, N. E. Petroulakis, W. Kellerer, and C. M. Machuca, "Incentives for a softwarization of wind park communication networks," *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 138–144, May 2019.

[18] (2016). *Data Center Power Equipment Thermal Guidelines and Best Practices*. [Online]. Available: https://www.ashrae.org/File%20Library/Technical%20Resources/Bookstore/ASHRAETC0909PowerWhitePaper22June2016REVISED.pdf

[19] A. Clemm and T. Eckert, "Combining autonomic and intent-based networking," in *Proc. NOMS - IEEE/IFIP Netw. Operations Manage. Symp.*, May 2023, pp. 1–6.

[20] A. Leivadeas and M. Falkner, "Autonomous network assurance in intent based networking: Vision and challenges," in *Proc. 32nd Int. Conf. Comput. Commun. Netw. (ICCCN)*, Jan. 2023, pp. 1–10.

[21] J. Wang, L. Guo, C. Zhou, Z. Chu, J. Wu, Q. Qi, Z. Zhuang, H. Sun, and J. Liao, "Revolutionizing network autonomy with intent-based network digital twins: Architecture, challenges, and innovations," *IEEE Commun. Mag.*, vol. 63, no. 8, pp. 85–91, Aug. 2025.

[22] Y. Njah, A. Leivadeas, and M. Falkner, "An AI-driven intent-based network architecture," *IEEE Commun. Mag.*, vol. 63, no. 4, pp. 146–153, Apr. 2025.

[23] A. Mwangi, K. Sundsgaard, J. A. L. Vilaplana, K. V. Vilerá, and G. Yang, "A system-based framework for optimal sensor placement in smart grids," in *Proc. IEEE Belgrade PowerTech*, Jun. 2023, pp. 1–6.

[24] M. C. Huebscher and J. A. McCann, "A survey of autonomic computing—Degrees, models, and applications," *ACM Comput. Surveys*, vol. 40, no. 3, pp. 1–28, Aug. 2008.

[25] R. Chaparadza, T. Ben Meriem, B. Radier, S. Szott, M. Wodczak, A. Prakash, J. Ding, S. Soulhi, and A. Mihailovic, "Implementation guide for the ETSI AFI GANA model: A standardized reference model for autonomic networking, cognitive networking and self-management," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, Dec. 2013, pp. 935–940.

[26] M. Xu, R. Liu, L. Tahvildari, and M. Stoodley, "An educational course on self-adaptive systems using ibm technologies," in *Proc. 33rd Annu. Int. Conf. Comput. Sci. Softw. Eng.*, 2023, pp. 143–148.

[27] E. Rutten, N. Marchand, and D. Simon, "Feedback control as MAPE-K loop in autonomic computing," in *Software Engineering for Self-Adaptive Systems III. Assurances: International Seminar*. Cham, Switzerland: Springer, 2017, pp. 349–373.

[28] B. P. Sanwouo Chekam, C. Quinton, and P. Temple, "Breaking the loop: AWARE is the new MAPE-K," in *Proc. 33rd ACM Int. Conf. Found. Softw. Eng.*, Jun. 2025, pp. 626–630.

[29] K. Mehmood, K. Kralevska, and D. Palma, "Intent-driven autonomous network and service management in future cellular networks: A structured literature review," *Comput. Netw.*, vol. 220, Jan. 2023, Art. no. 109477.

[30] A. Mwangi, N. Kabbara, P. Coudray, M. Gryning, and M. Gibescu, "Investigating the dependability of software-defined IIoT-edge networks for next-generation offshore wind farms," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 6, pp. 6126–6139, Dec. 2024.

[31] S. Jaberi, J. W. Atwood, and J. Paquet, "ASSL as an intent expression language for autonomic intent-driven networking," in *Proc. 19th Int. Conf. Netw. Service Manage. (CNSM)*, Oct. 2023, pp. 1–5.

[32] A. Mwangi, L. Navarro-Hilfiker, L. Brewka, M. Gryning, E. Fumagalli, and M. Gibescu, "A threshold-triggered deep Q-network-based framework for self-healing in autonomic software-defined IIoT-edge networks," *IEEE Trans. Netw. Service Manage.*, vol. 23, pp. 1297–1311, 2026.

[33] A. Mwangi, A. Presekal, M. Gryning, E. Fumagalli, M. Gibescu, and A. Stefanov, "ASDA: An autonomic self-defense agent for cyber-resilient offshore wind software-defined industrial networks," *IEEE Trans. Ind. Inform.*, pp. 1–12, 2026.
[34] J. P. G. Sterbenz, D. Hutchison, E. K. Çetinkaya, A. Jabbar, J. P. Rohrer, M. Schöller, and P. Smith, "Resilience and survivability in communication networks: Strategies, principles, and survey of disciplines," *Comput. Netw.*, vol. 54, no. 8, pp. 1245–1265, Jun. 2010.
[35] J. Bozic, M. Piesche, K. Reti, and T. Sandner, "A framework for cyber resilience assessment in industrial control systems," *Comput. Secur.*, vol. 77, pp. 648–664, Feb. 2018.
[36] M. Bruneau, S. E. Chang, R. T. Eguchi, G. C. Lee, T. D. O'Rourke, A. M. Reinhorn, M. Shinozuka, K. Tierney, W. A. Wallace, and D. von Winterfeldt, "A framework to quantitatively assess and enhance the seismic resilience of communities," *Earthq. Spectra*, vol. 19, no. 4, pp. 733–752, Nov. 2003.

AGRIPPINA MWANGI received the B.Sc. degree in electrical and electronic engineering from the Dedan Kimathi University of Technology, Kenya, and the M.Sc. degree in electrical and computer engineering from Carnegie Mellon University, Pittsburgh, PA, USA. She is currently pursuing the Ph.D. degree with the Energy and Resources Group, Copernicus Institute of Sustainable Development, Utrecht University, The Netherlands. Her research interests include IIoT-edge, SDN/NFV, network automation, cyber resilience, performance modeling, and offshore wind.

MIKKEL GRYNING received the M.Sc. and Ph.D. degrees in electrical engineering from the Technical University of Denmark. He is currently the Chief System Design Specialist at Ørsted. His research interests include power system stability robustness evaluation, hybrid power plant integration and design, and communication systems for offshore wind power plants.

ELENA FUMAGALLI received the M.Sc. degree in nuclear engineering from the Politecnico di Milano, Italy, and the Ph.D. degree in energy engineering from the Università di Padova, Italy. She is currently an Associate Professor with the Section Energy and Resources, Department of Sustainable Development, Utrecht University. Her research interests include electricity market design, network regulation, and smart grids.

MADELEINE GIBESCU (Member, IEEE) received the Ph.D. degree in electrical engineering from the University of Washington, Seattle, WA, USA, in 2003. Since 2004, she has been working in The Netherlands in various academic capacities at TU Delft, TU Eindhoven, and Utrecht University, leading research in smart grids and system integration of wind energy and solar photovoltaics. She is currently a Full Professor of integration of intermittent renewable energy with the Copernicus Institute of Sustainable Development, Faculty of Geosciences, Utrecht University. Her research interests include the modeling and simulation of power systems and electricity markets with a large penetration of renewable energy sources.